

Monocular Visual SLAM Using Custom Video Input

Ma Qihao
Southern Illinois University Edwardsville
ME/ECE/CS 462 – Robotic Vision

April 2025

Contents

1	Introduction	2
2	SLAM Example Using the TUM Dataset	2
2.1	Overview of the Example	2
2.2	Key Results	2
2.3	Discussion	2
3	SLAM Using Custom Video Input	3
3.1	Video Frame Processing	3
3.2	Map Initialization and Feature Matching	3
3.3	Feature Point Detection	3
3.4	Trajectory and Point Cloud Reconstruction	3
3.5	Discussion	4
3.6	Camera Calibration	4
4	Reflection and Future Work	4
5	Project Files	5

1. Introduction

Simultaneous Localization and Mapping (SLAM) helps robots find their location and create a map of the area at the same time. Visual SLAM uses a camera. Monocular SLAM uses one camera, which is cheaper and simpler. But it is harder to get depth and scale.

This project uses MATLAB to run monocular SLAM. First, we test with a public dataset. Then, we use a video we recorded ourselves. We focus on detecting features, finding the camera path, building the map, and checking how well it works.

2. SLAM Example Using the TUM Dataset

2.1. Overview of the Example

The example from MathWorks follows the idea of ORB-SLAM. It uses the TUM RGB-D dataset. The main steps are:

- Load a group of images
- Detect and extract ORB features
- Match features between frames
- Estimate camera poses
- Build 3D points by triangulation
- Use loop closure and optimization to fix errors

The example uses MATLAB's tools, which are easy to test and run.

2.2. Key Results

Figure 1 shows that the camera path is close to the real one. The error is small.

Figure 2 shows how features match in the beginning.

Figure 3 shows the 3D points and camera path, with loop closure.

Figure 4 shows the detected ORB features.

2.3. Discussion

The figures show that ORB features work well. In Figure 1, the camera path matches the real one. In Figures 2 and 4, the features are found clearly. Figure 3 shows that optimization makes the map and path better.

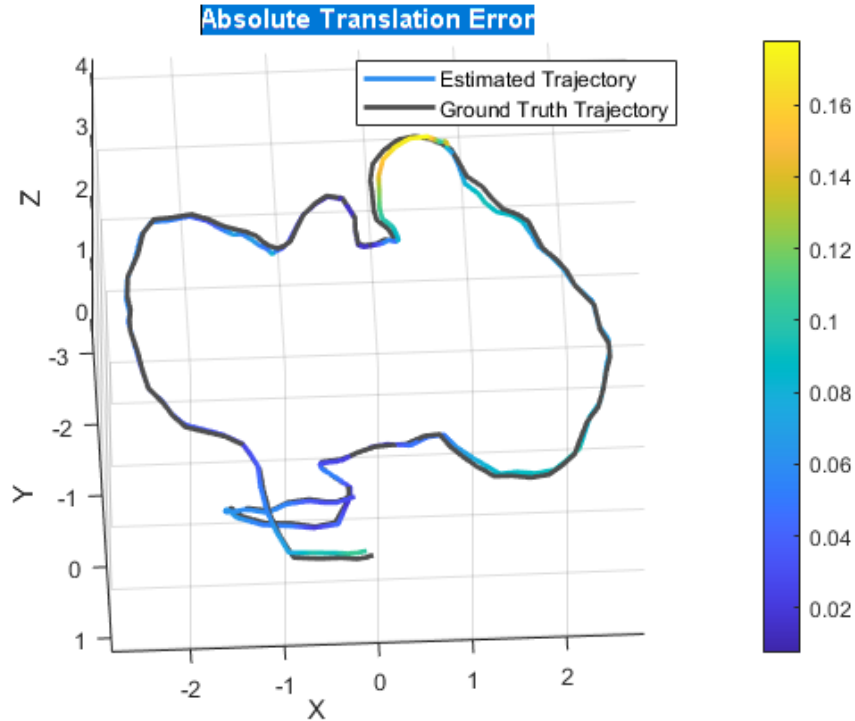


Figure 1: Absolute Translation Error: Estimated vs Ground Truth Trajectory

3. SLAM Using Custom Video Input

3.1. Video Frame Processing

I recorded a video called `video2.mp4`. To prepare the video:

- Read frames with `VideoReader`
- Resize each frame to 1080×1920
- Save frames in a cell array

This helps keep all images the same size and format.

3.2. Map Initialization and Feature Matching

Figure 5 shows matched features from the first frames. These matches help find the camera motion and 3D points.

3.3. Feature Point Detection

ORB detects key points. Figure 6 shows the detected features.

3.4. Trajectory and Point Cloud Reconstruction

Figure 7 shows the camera path and 3D map points from the custom video.

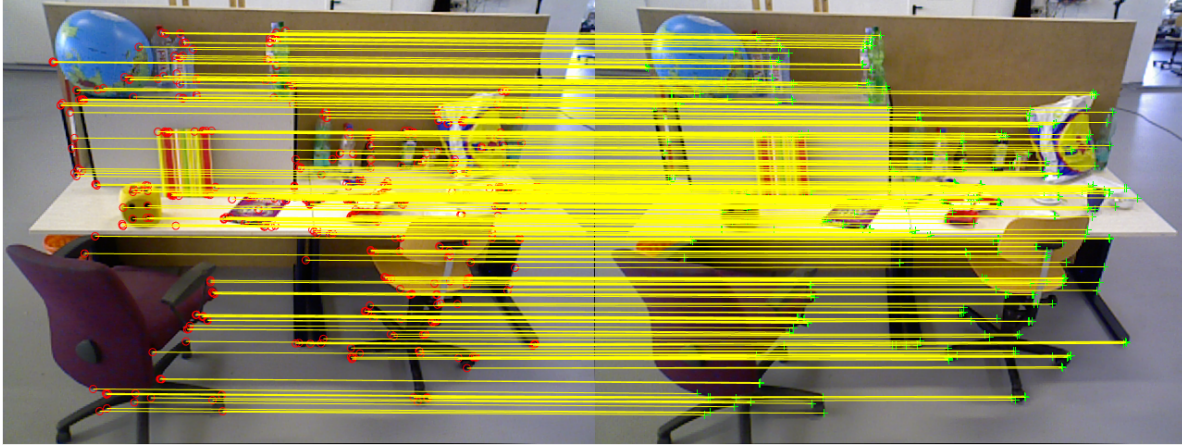


Figure 2: Feature Matching Between Keyframes During Initialization

3.5. Discussion

This test shows the SLAM system works in a new place. Even though the room has low light and few textures, the system still:

- Finds and tracks features
- Estimates the camera path
- Builds a 3D map

Figures 5, 6, and 7 show the results.

3.6. Camera Calibration

We used MATLAB to get the camera intrinsics. This is important for good SLAM results.

- Focal Length: $[1170.7, 1172.6]$
- Principal Point: $[992.0, 589.5]$
- Image Size: 1920×1080

4. Reflection and Future Work

This project shows monocular SLAM works with both a dataset and a real video. We learned that camera calibration, frame quality, and feature matching are important.

Later, we can:

- Add IMU data to help during fast motion
- Use ORB-SLAM3 or OpenVSLAM to get real-time results
- Try other feature detectors like SuperPoint or SIFT

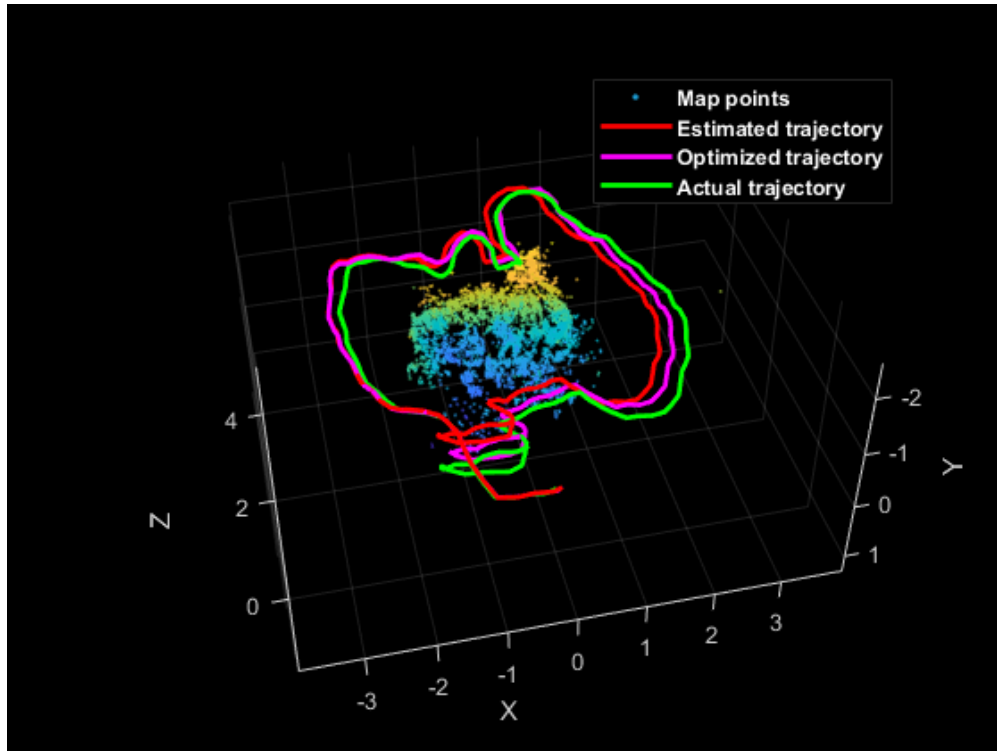


Figure 3: Map Points and Trajectory Visualization: Estimated, Optimized, and Ground Truth

5. Project Files

All code, images, and video are on GitHub:

https://github.com/Qiuhaoma/462_SLAM_project

References

- [1] MathWorks. Monocular Visual Simultaneous Localization and Mapping.
<https://www.mathworks.com/help/vision/ug/monocular-visual-simultaneous-localization-a.html>
- [2] TUM RGB-D Dataset.
<https://vision.in.tum.de/data/datasets/rgbd-dataset>

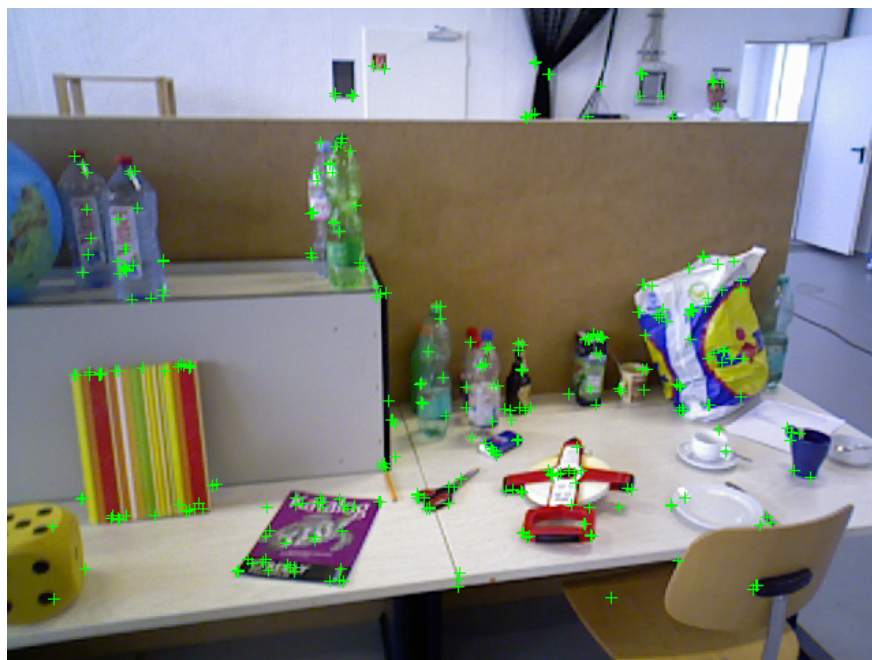


Figure 4: Feature Points Used in Initial 3D Map Reconstruction

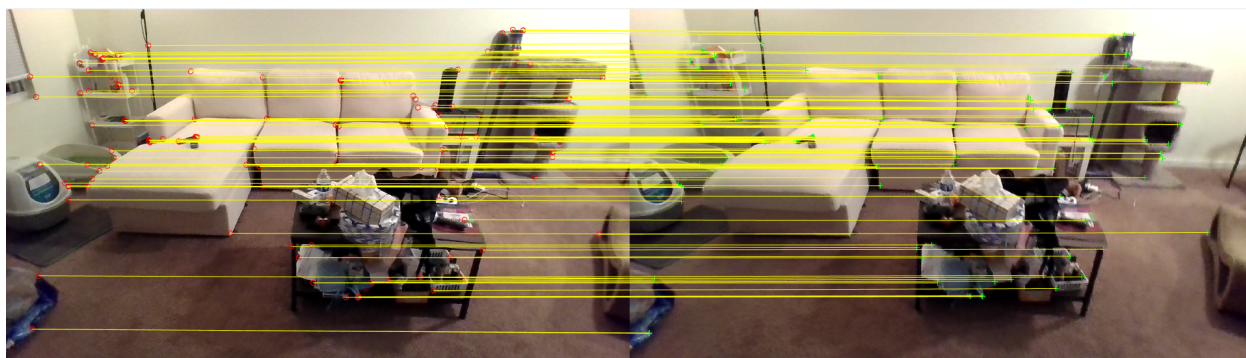


Figure 5: Feature Matching Between Initial Frames (Custom Video)

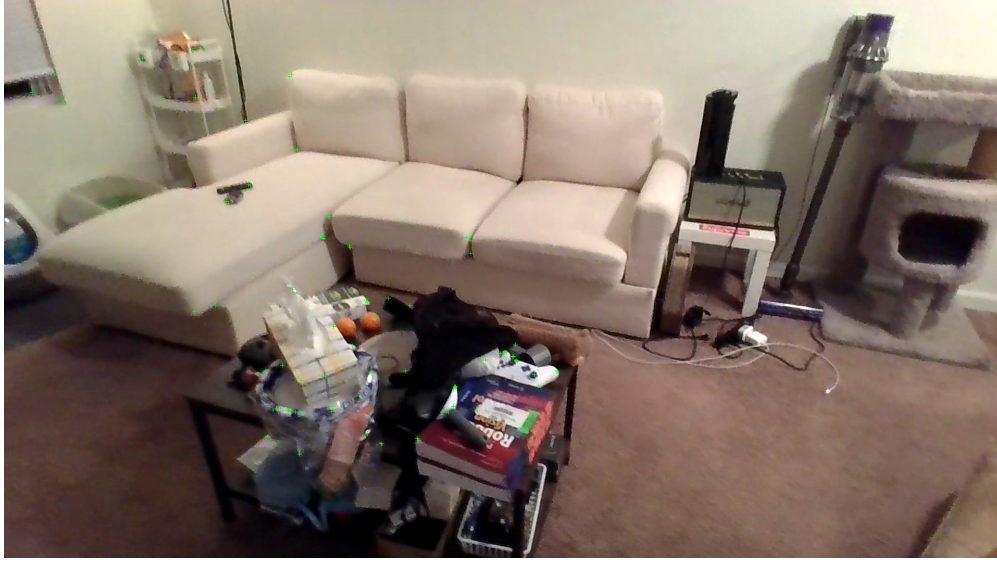


Figure 6: Feature Points Detected on a Frame from the Custom Video

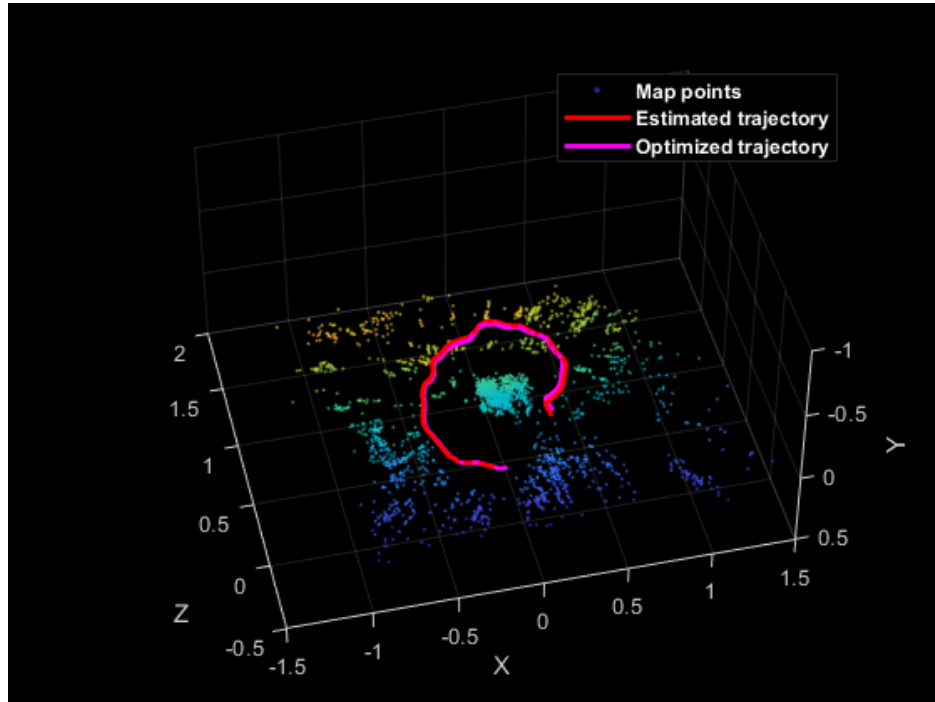


Figure 7: Map Points and Estimated, Optimized, and Ground Truth Camera Trajectories (Custom Video)